

1.1. Диаграмма классов (class diagram)

Литература

1. Леоненков А. В. Самоучитель UML 2. — СПб.: БХВ-Петербург, 2007. — 576 с.: ил
2. Буч Г., Рамбо Д., Якобсон И. Язык UML. Руководство пользователя. 2-е изд.: Пер. с англ. Мухин Н. — М.: ДМК Пресс, 2006. — 496 с.: ил.
3. UML: первое знакомство / Бабич А.В. — М.: Национальный Открытый Университет «ИНТУИТ», 2016 (Основы информационных технологий).
4. Гурьянов В. И. Имитационное моделирование на UML SP: монография / В.И. Гурьянов. — Чебоксары : Филиал СПбГЭУ в г. Чебоксары, 2014. — 135 с.

1.1.1. Общие сведения

Диаграмма классов (class diagram) — диаграмма, предназначенная для представления модели статической структуры программной системы в терминологии классов объектно-ориентированного программирования.

Диаграмма классов служит для представления статической структуры модели системы в терминологии классов объектно-ориентированного программирования. Диаграмма классов может отражать, в частности, различные взаимосвязи между отдельными сущностями предметной области, такими как объекты и подсистемы, а также описывает их внутреннюю структуру (поля, методы...) и типы отношений (наследование, реализация интерфейсов ...). На данной диаграмме не указывается информация о временных аспектах функционирования системы. С этой точки зрения диаграмма классов является дальнейшим развитием концептуальной модели проектируемой системы. На этом этапе принципиально знание ООП подхода и паттернов проектирования.



Рис. 4.1. Варианты графического изображения класса на диаграмме классов:
а — в виде прямоугольника; б — в виде прямоугольника с секциями;
в — без секции атрибутов класса; г — активный класс



Рис. 4.2. Примеры графического изображения конкретных классов:
 а — для класса *Линия* указаны атрибуты; б — для класса *Окно* указаны операции;
 в — для класса *Счет* указано исключение

Каждому атрибуту класса в языке UML 2.0 соответствует отдельная строка текста, которая в нотации БНФ имеет следующий формат:

```
<атрибут> ::= [<видимость>] ['/'<имя> ':'<тип атрибута>]
['['<кратность>']'] ['='<значение по умолчанию>]
['{'<модификатор атрибута> ','<модификатор атрибута>']* '}'
```

Здесь:

- □ <видимость> ::= '+' | '-' | '#' | '~'. Другими словами, видимость (visibility) может принимать одно из 4-х возможных значений и отображаться либо посредством специального символа, либо соответствующего ключевого слова;
 Вид видимости является перечислением следующих значений литералов:
 - □ public (общедоступный). Общедоступный элемент является видимым всеми элементами, который имеют доступ к содержимому пространства имен, который им владеет;
 - □ private (закрытый). Закрытый элемент является видимым только внутри пространства имен, который им владеет;
 - □ protected (защищенный). Защищенный элемент является видимым для элементов, которые имеют отношение обобщения с пространством имен, который им владеет;
 - □ package (пакет). Элемент, помеченный как имеющий пакетную видимость, является видимым всеми элементами в ближайшем охватывающем пакете. За пределами ближайшего охватывающего пакета элемент, помеченный как имеющий пакетную видимость, не является видимым.

Вместо литерала видимости в языке UML 2.0 допускается записывать соответствующий символ видимости:

- + — общедоступный;
- - — закрытый;
- # — защищенный;
- ~ — пакетный.

- □ '/' означает, что атрибут является производным (derive). Значение производного атрибута может быть вычислено на основе значений других атрибутов этого или других классов. Поэтому данный атрибут называют иногда вычислимым. При использовании производных атрибутов разработчик должен явно указать процедуру или операцию для вычисления их значений;

- □ <имя> (name) представляет собой строку текста, которая используется в качестве идентификатора соответствующего атрибута и поэтому должна быть уникальной в пределах данного класса. Имя атрибута является единственным обязательным элементом в обозначении атрибута, должно начинаться со строчной (малой) буквы и, как правило, не должно содержать пробелов;
- □ <тип атрибута> (attribute type) есть имя классификатора, который является типом данного атрибута. Тип атрибута представляет собой имя некоторого типа данных, определенного или в пакете Типы данных языка UML 2.0, или самим разработчиком. Типы атрибутов классов часто соответствуют основным типам данных того языка программирования, который предполагается использовать для реализации данной модели. В общем случае тип атрибута записывается строкой текста, имеющей осмысленное значение в пределах пакета или модели, к которому относится рассматриваемый класс. Типу атрибута должно предшествовать двоеточие;
- □ <кратность> (multiplicity) атрибута характеризует общее количество конкретных значений для атрибута, которые могут быть заданы для объектов данного класса; Спецификация кратности в нотации БНФ имеет следующий формат:

```

<кратность> ::= <диапазон-кратности> [ '{' <указатель-упорядоченности>
[ ',' <указатель-уникальности> ] '}' ]
<диапазон-кратности> ::= [ <нижняя-граница> '..' ] <верхняя-граница>
<нижняя-граница> ::= <целое число> | <спецификация значения>
<верхняя-граница> ::= '*' | <спецификация значения>
<указатель-упорядоченности> ::= 'ordered' | 'unordered'
<указатель-уникальности> ::= 'unique' | 'nonunique'

```

Для представления неограниченной (бесконечной) верхней границы может быть использован символ "звездочка" * как часть спецификации кратности.

Спецификация кратности для атрибута дополнительно помещается в квадратные скобки [].

- □ <значение по умолчанию> (default) — некоторое выражение, которое служит для задания начального значения или значений данного атрибута в момент создания отдельного экземпляра соответствующего класса. Конкретное значение по умолчанию должно соответствовать типу данного атрибута. Если этот терм не указан, то значение атрибута на момент создания нового экземпляра класса не определено. С другой стороны, конструктор соответствующего объекта может переопределять исходное значение в процессе выполнения программы, если в этом возникает необходимость;
- □ <модификатор атрибута> (attribute modifier) представляет собой текстовое выражение, которое придает дополнительную семантику данному атрибуту. При этом набор возможных модификаторов атрибутов в языке UML 2.0 фиксирован и может быть представлен в следующем виде (БНФ):

```

<модификатор атрибута> ::= 'readOnly' | 'union' | 'subsets'
<имя атрибута> | 'redefines' <имя атрибута> | 'ordered' |
'unique' | <ограничение атрибута>

```

Назначение указанных модификаторов описываются в табл. 4.1.

Таблица 4.1. Назначение модификаторов атрибутов в языке UML 2.0

Модификатор атрибута	Назначение модификатора
readOnly	Атрибут является атрибутом только для чтения
union	Атрибут является производным объединением его подмножеств
subsets <имя атрибута>	Атрибут является собственным подмножеством атрибута с именем <имя атрибута>
redefines <имя атрибута>	Атрибут переопределяет некоторый наследуемый атрибут с именем <имя атрибута>
ordered	Значения атрибута являются упорядоченными. Этот порядок означает, что существует отображение из множества положительных целых чисел в элементы этой коллекции значений. Если атрибут не является многозначным, то значение для ordered не имеет семантического эффекта. При отсутствии этого модификатора атрибут специфицируется как неупорядоченный, и никаких предположений нельзя сделать относительно порядка его возможных значений
unique	Значения многозначного атрибута не могут иметь дубликатов, т. е. повторяться. Предполагается, что кратность соответствующего атрибута должна быть больше 1. Если атрибут не является многозначным, то значение unique не имеет семантического эффекта
<ограничение атрибута>	Выражение, которое специфицирует некоторое ограничение, применяемое к данному атрибуту

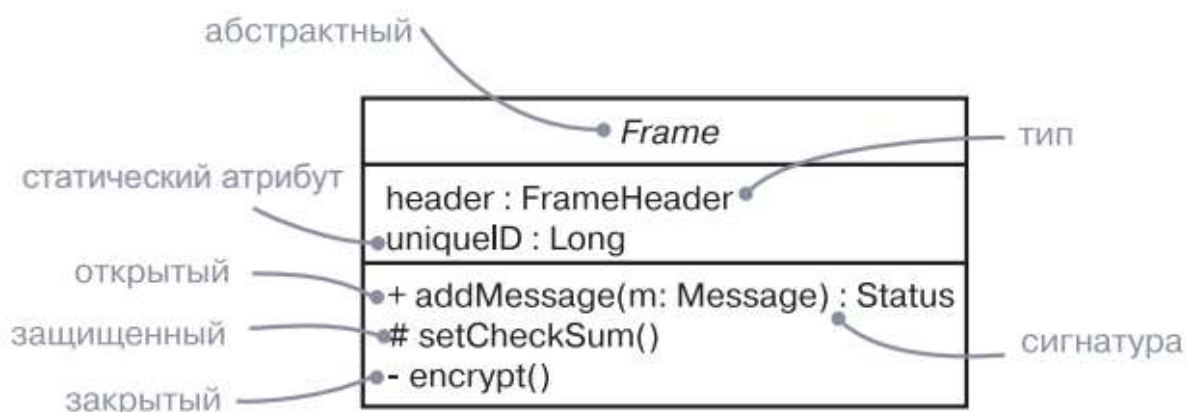


Рис. 9.1. Расширенные классы

1.1.2. Отношения между классами

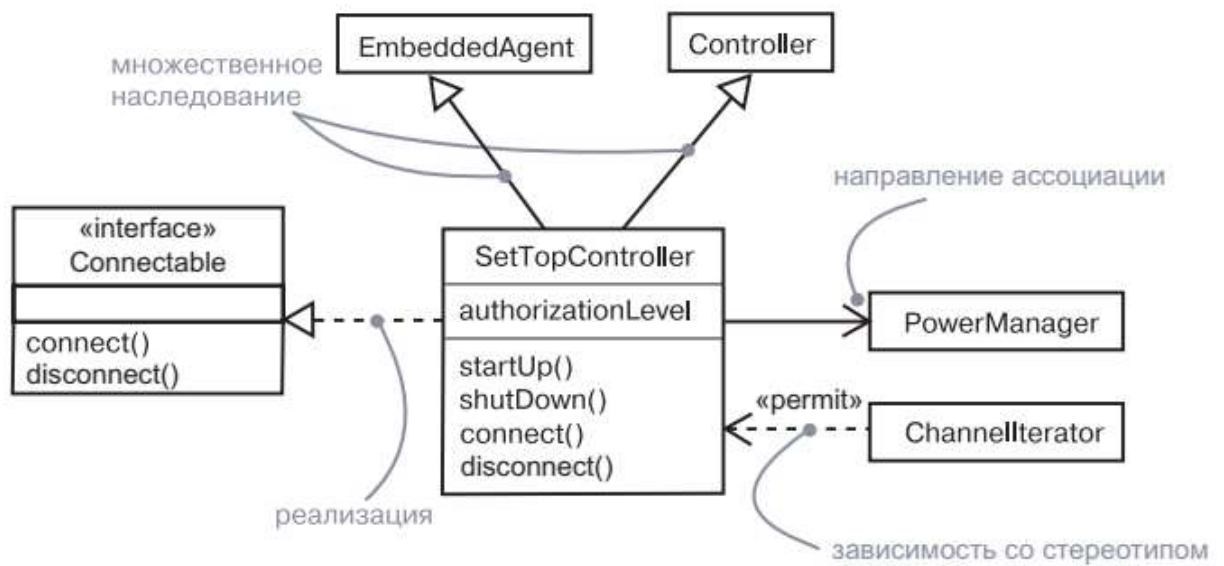


Рис. 10.1. Расширенные связи

1.1.2.1. Ассоциация

Ассоциация (association) — произвольное отношение или взаимосвязь между классами.

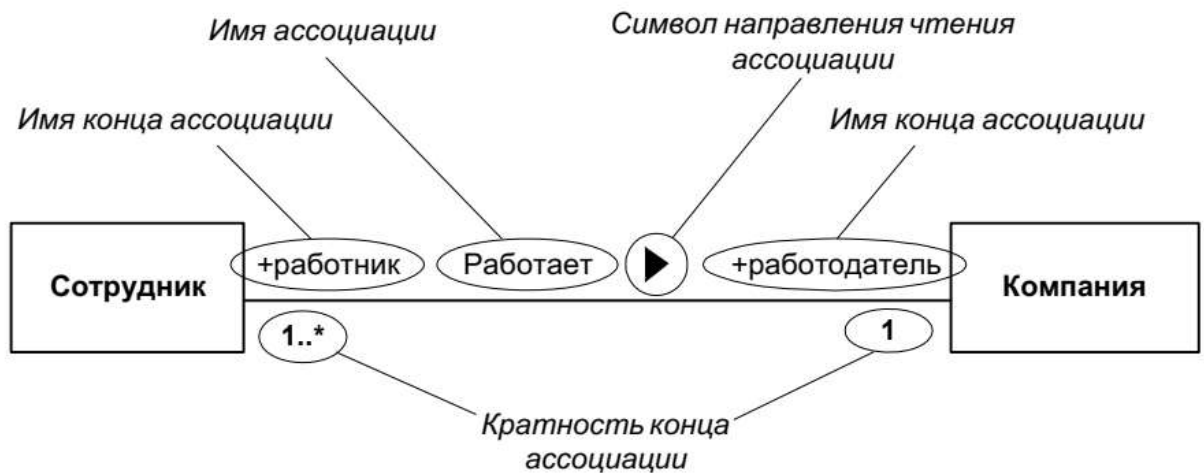


Рис. 4.3. Графическое изображение ненаправленной бинарной ассоциации между классами



Рис. 4.4. Графическое изображение: а — бинарной ассоциации с навигацией; б — эквивалентное ему представление класса с атрибутом

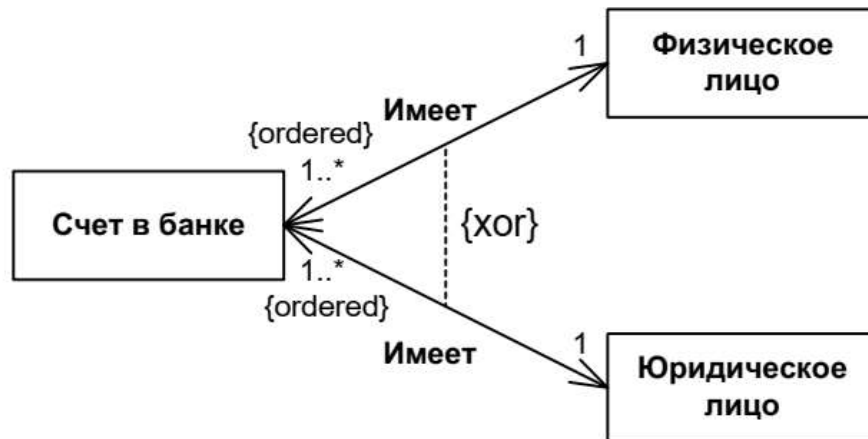


Рис. 4.6. Графическое изображение исключяющей ассоциации между тремя классами

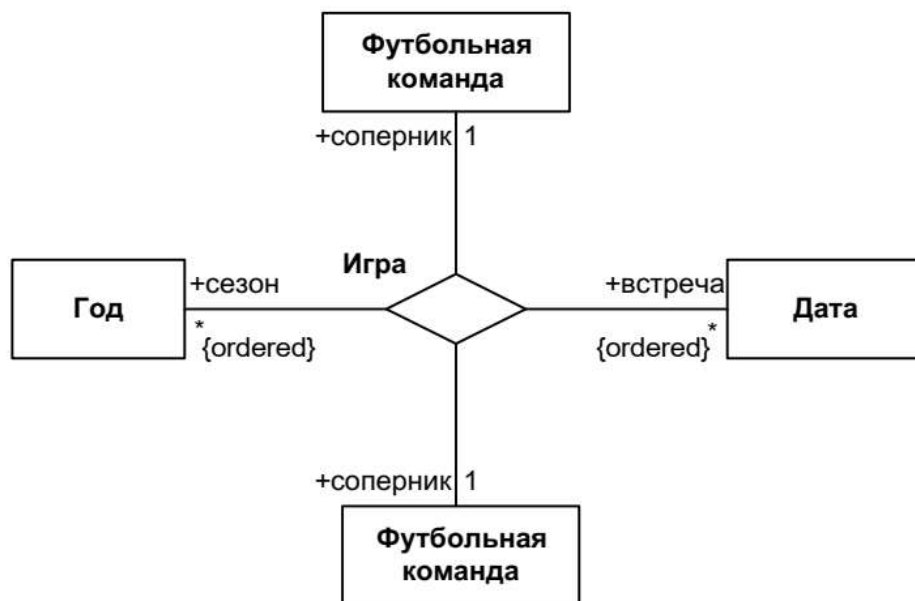


Рис. 4.7. Графическое изображение 4-арной ассоциации

1.1.2.2. Ассоциация-класс

Ассоциация-класс (association class) — элемент модели, который имеет свойства как ассоциации, так и класса, и предназначенный для спецификации дополнительных свойств ассоциации в форме атрибутов и, возможно, операций класса.



Рис. 4.8. Графическое изображение ассоциации класса

Квалификатор

Квалификатор (qualifier) объявляет разбиение множества ассоциированных экземпляров относительно экземпляра на квалифицированном конце ассоциации.



Рис. 4.9. Графическое изображение квалификатора:
а — для ассоциации; б — для композиции



Рис. 10.7. Ассоциации-классы

1.1.2.3. Обобщение

Обобщение (generalization) — таксономическое отношение между более общим классификатором (родителем или предком) и более специальным классификатором (дочерним или потомком).

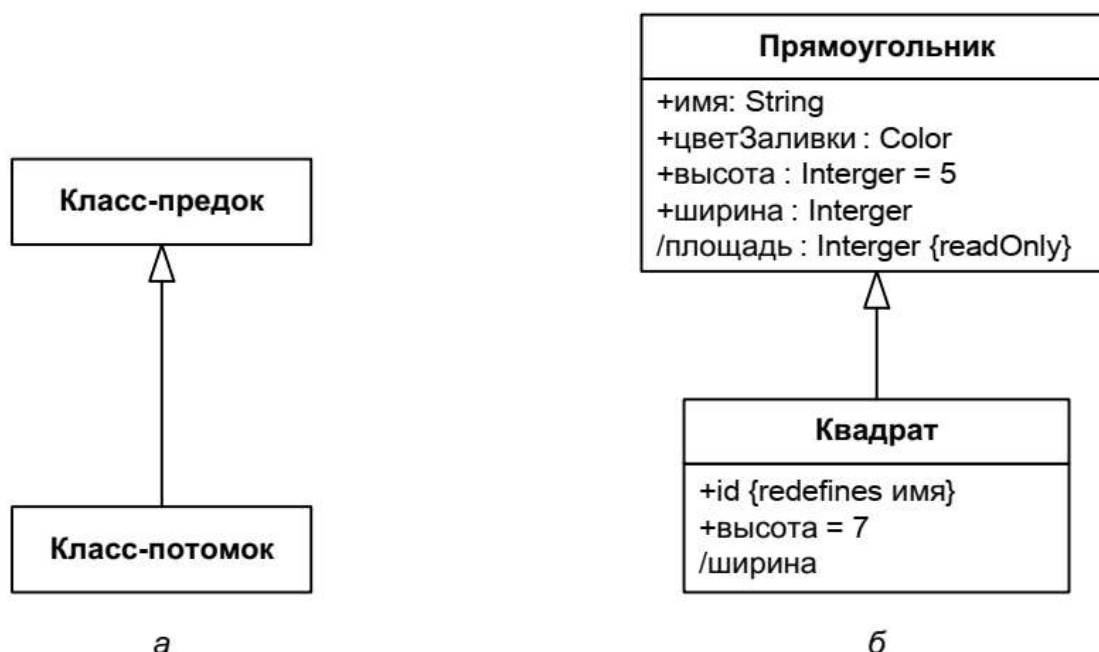


Рис. 4.10. Графическое изображение отношения обобщения в языке UML 2.0: а — свернутое представление классов; б — развернутое представление классов

Комментарий к рисунку: Для изображенного справа отношения обобщения (рис. 4.10, б) класс Квадрат наследует от класса Прямоугольник все атрибуты. Дополнительный атрибут id класса Квадрат переопределяет атрибут имя класса Прямоугольник, а атрибут высота класса Квадрат имеет по умолчанию значение 7, вместо значения 5 в родительском классе. Атрибут ширина в классе Квадрат является производным, поскольку его значение всегда равно значению атрибута высота. Хотя атрибуты цвет Заливки и площадь в классе Квадрат не указаны, они также принадлежат этому классу. Их спецификация полностью идентична описанию в родительском классе, что следует из семантики отношения обобщения.

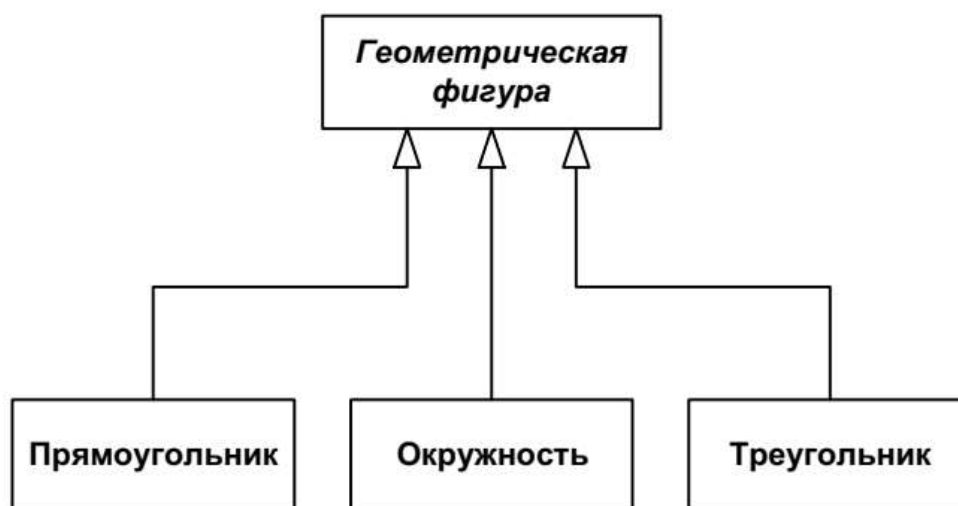


Рис. 4.11. Пример графического изображения отношения обобщения для нескольких классов-потомков

Комментарий к рисунку: Класс *Геометрическая фигура* (курсив обозначает абстрактный класс) может выступать в качестве родительского класса для классов-потомков, соответствующих конкретным геометрическим фигурам, таким как Прямоугольник, Окружность, Эллипс и др.



Рис. 4.12. Альтернативный вариант графического изображения отношения обобщения классов (см. рис. 4.11) в "стиле совместно используемой цели"

1.1.2.4. Множество обобщения

Множество обобщения (generalization set) — элемент модели, экземпляры которого определяют коллекции подмножеств отношения обобщения.



Рис. 4.13. Диаграмма классов для двух множеств обобщения

Примечание к рисунку: Человек является абстрактным классом, поскольку объект Человек не может существовать без того, чтобы быть или Женщиной, или Мужчиной.

Множество обобщения может содержать строку текста, указывающую на некоторые специальные свойства этого отношения. Этот текст будет относиться ко всем линиям обобщения, которые идут к классам-потомкам данного множества обобщения. Чтобы указать, покрывает или нет множество обобщения, а также пересекаются или нет его элементы, каждое множество должно быть помечено одним из следующих ограничений:

- ☐ {complete, disjoint} — означает, что данное множество обобщения является покрывающим и его специальные классы не имеют общих экземпляров;
- ☐ {incomplete, disjoint} — означает, что данное множество обобщения не является покрывающим и его специальные классы не имеют общих экземпляров (предполагается по умолчанию);

- {complete, overlapping} — означает, что данное множество обобщения является покрывающим и его специальные классы имеют общие экземпляры;
- {incomplete, overlapping} — означает, что данное множество обобщения не является покрывающим и его специальные классы имеют общие экземпляры.

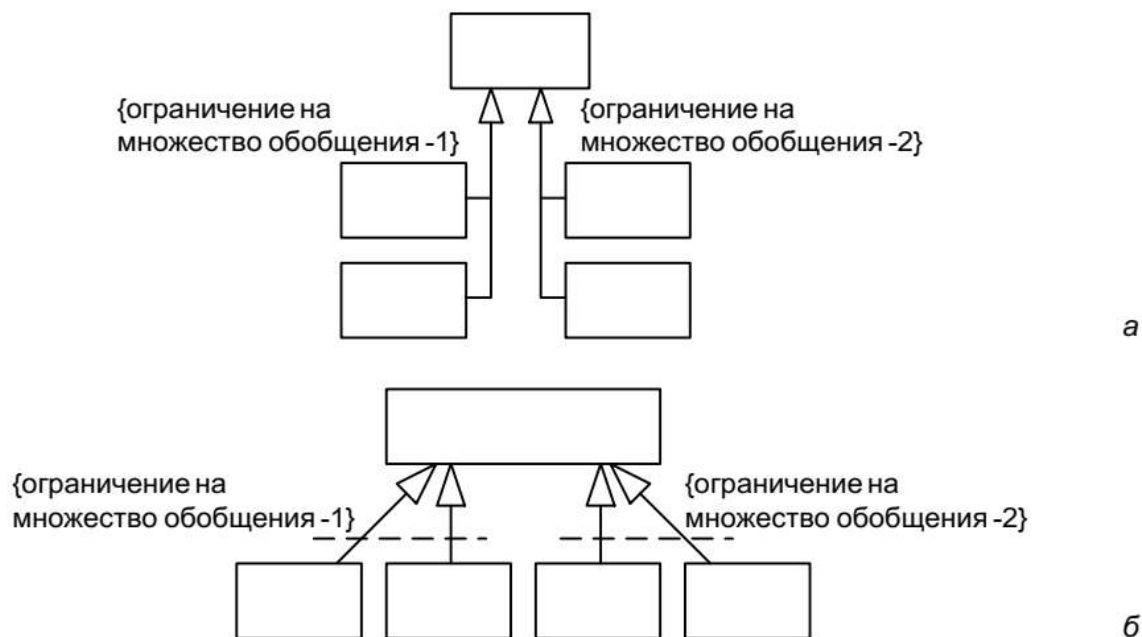


Рис. 4.14. Варианты нотации для ограничения на множество обобщения:
а — с помощью общей стрелки обобщения; б — с помощью пунктирной линии



Рис. 4.15. Вариант графического изображения отношения обобщения классов с использованием ограничений на множество обобщения

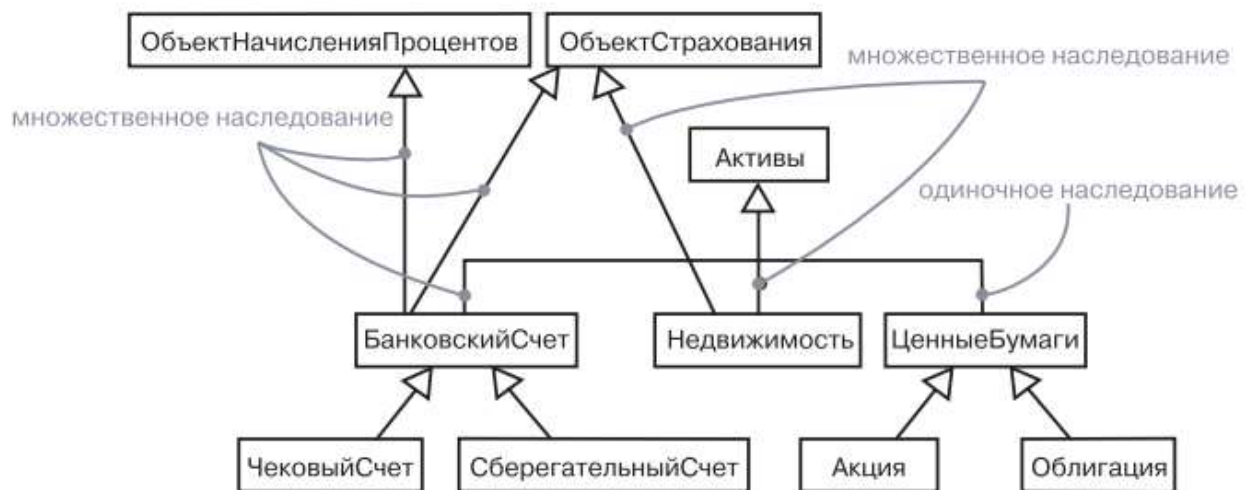


Рис. 10.2. Множественное наследование

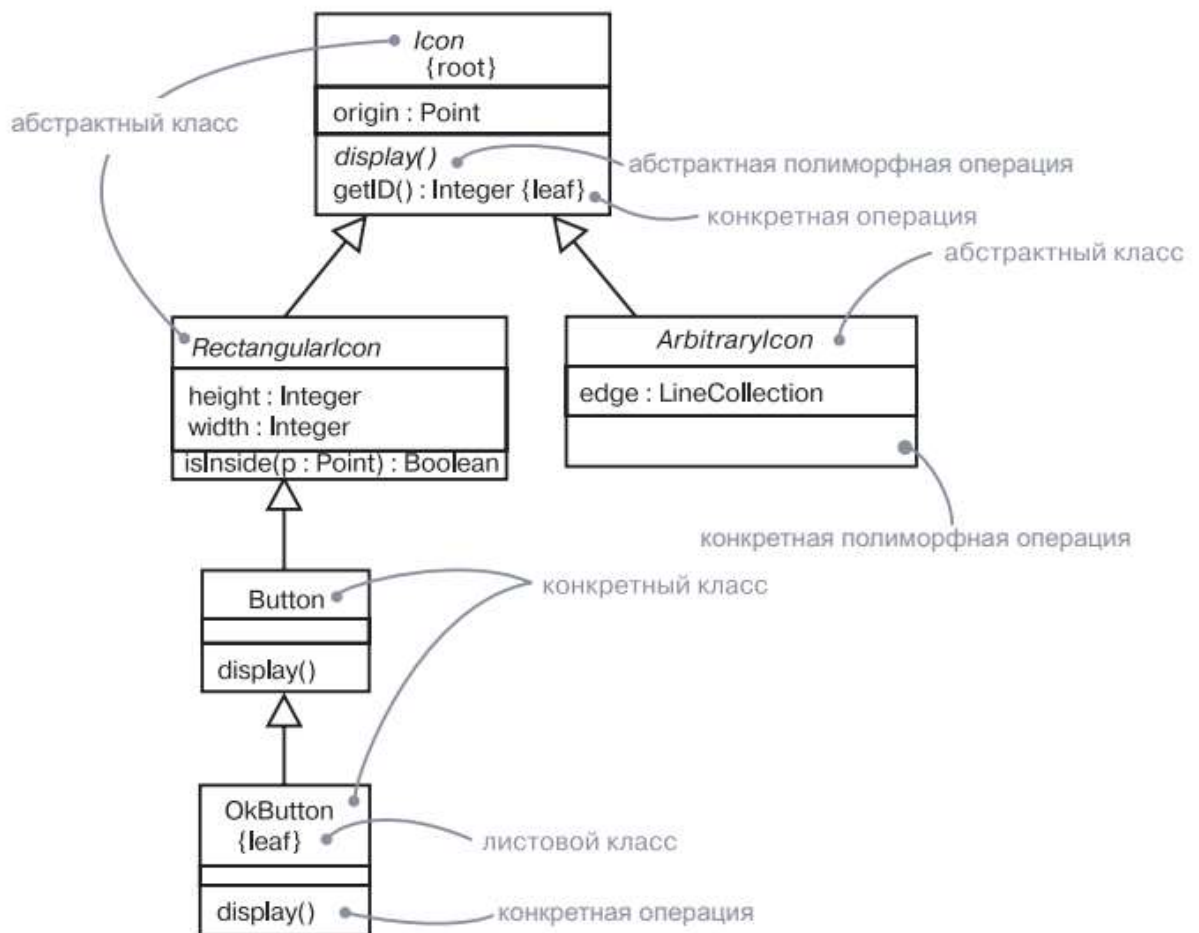


Рис. 9.5. Абстрактные и конкретные классы и операции

1.1.2.5. Агрегация

Агрегация (aggregation) — направленное отношение между двумя классами, предназначенное для представления ситуации, когда один из классов представляет собой некоторую сущность, которая включает в себя в качестве составных частей другие сущности.

Ромб агрегации по форме должен быть меньше, чем ромб в качестве нотации ассоциации. Этот ромб указывает на тот из классов, который представляет собой "класс-целое" или класс-контейнер. Остальные классы являются его "частями" (рис. 4.16)

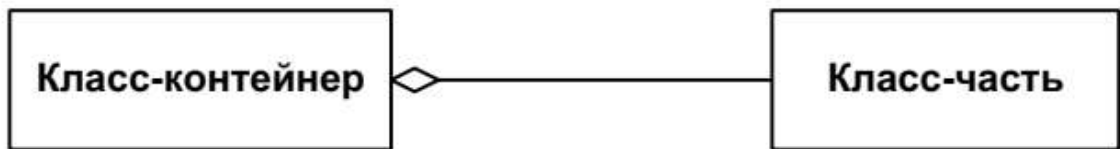


Рис. 4.16. Графическое изображение отношения агрегации

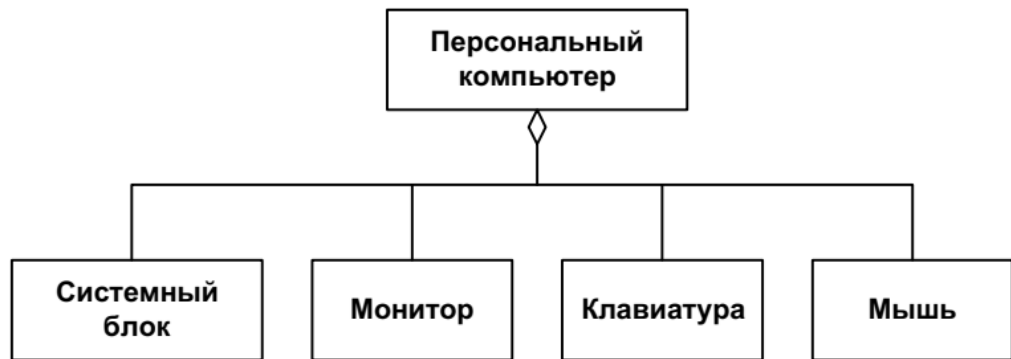


Рис. 4.17. Диаграмма классов для иллюстрации отношения агрегации на примере ПК

1.1.2.6. Композиция

Композиция (composition) или композитная агрегация предназначена для спецификации более сильной формы отношения "часть-целое", при которой с уничтожением объекта класса-контейнера уничтожаются и все объекты, являющиеся его составными частями.



Рис. 4.18. Графическое изображение отношения композиции

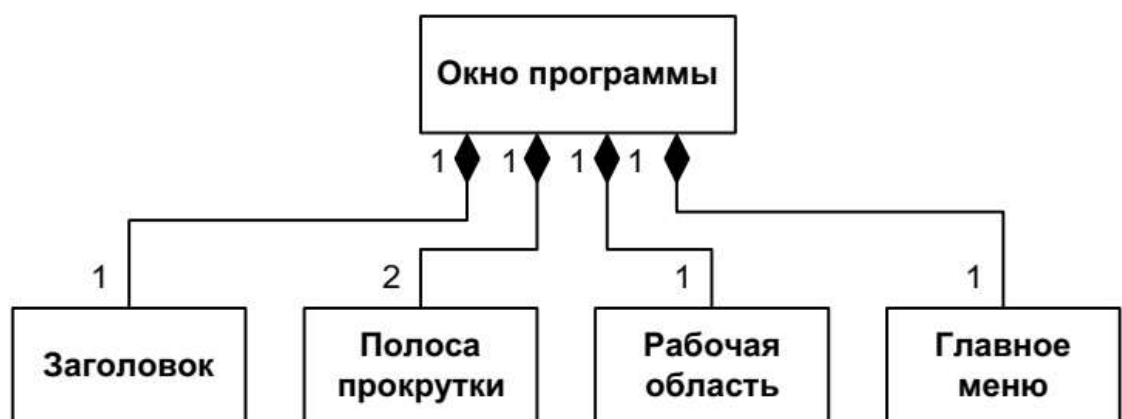


Рис. 4.19. Диаграмма классов для иллюстрации отношения композиции на примере класса-композиита *Окно программы*

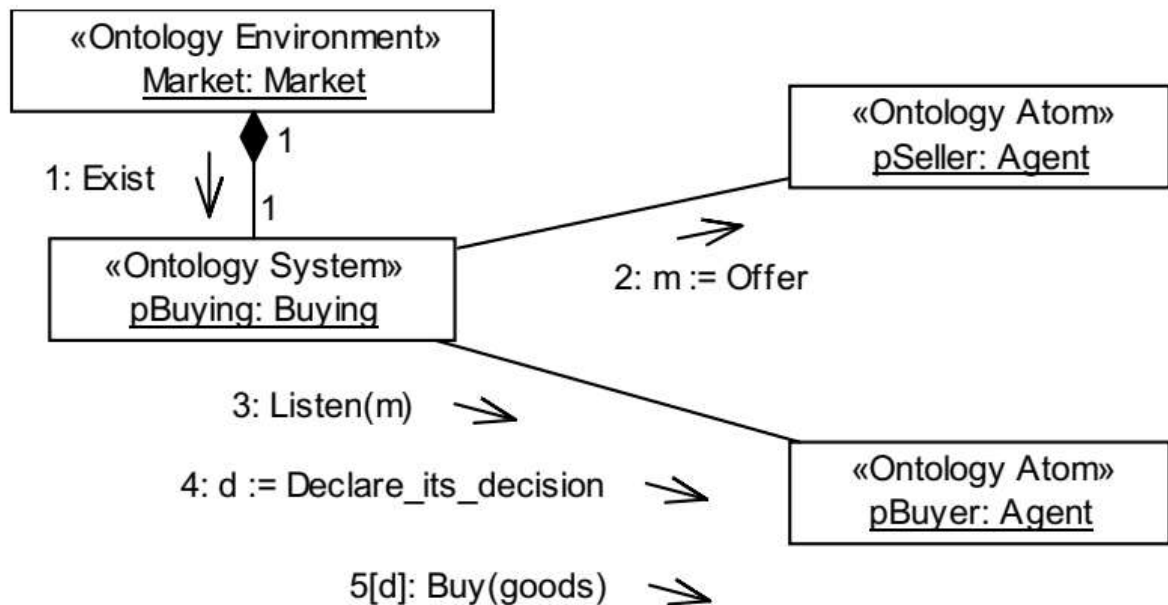


Рис. 6. Диаграмма кооперации

1.1.2.7. Зависимость

Зависимость (dependency) — отношение, предназначенное для описания ситуации, когда отдельному элементу или множеству элементов модели требуются другие элементы модели для своей спецификации или реализации.



Рис. 4.20. Графическое изображение отношения зависимости на диаграмме классов

1.1.2.8. Реализация

Реализация (realization) — специализированное отношение зависимости между двумя элементами модели, один из которых представляет некоторую спецификацию (поставщик), а другой представляет его реализацию (клиент)

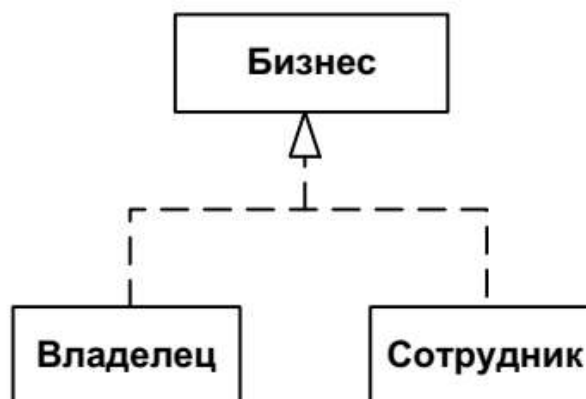


Рис. 4.21. Графическое изображение отношения реализации на диаграмме классов

Примечание к рисунку: класс Бизнес реализуется комбинацией классов Владелец и Сотрудник.

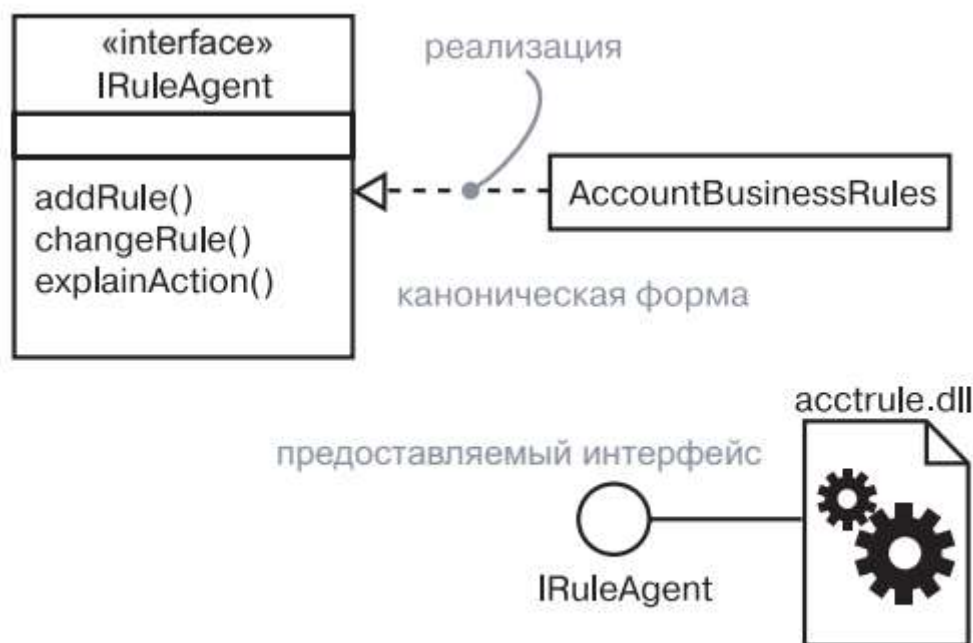


Рис. 10.8. Реализация интерфейса

1.1.2.9. Интерфейс

Интерфейс (interface) — вид класса, который представляет собой объявление множества общедоступных характеристик и обязанностей.



Рис. 4.22. Пример графического изображения требуемого (слева) и предоставляемого (справа) интерфейса



Рис. 4.23. Пример альтернативной нотации для графического изображения интерфейса для ситуации, изображенной на рис. 4.22

1.1.2.10. Шаблон

Шаблон (template) — классификатор, который в своем описании имеет несколько формальных параметров.

Общая нотация для формального параметра шаблона — строка, изображаемая внутри списка параметров шаблона:

$\langle \text{параметр-шаблона} \rangle ::= \langle \text{имя-параметра-шаблона} \rangle [':' \langle \text{вид-параметра} \rangle] ['=' \langle \text{по умолчанию} \rangle]$

где $\langle \text{вид-параметра} \rangle$ является именем метакласса для используемого элемента.

Синтаксис $\langle \text{имя-параметра-шаблона} \rangle$ зависит от вида параметризуемого элемента для данной подстановки параметра шаблона, а $\langle \text{по умолчанию} \rangle$ зависит от вида элемента.

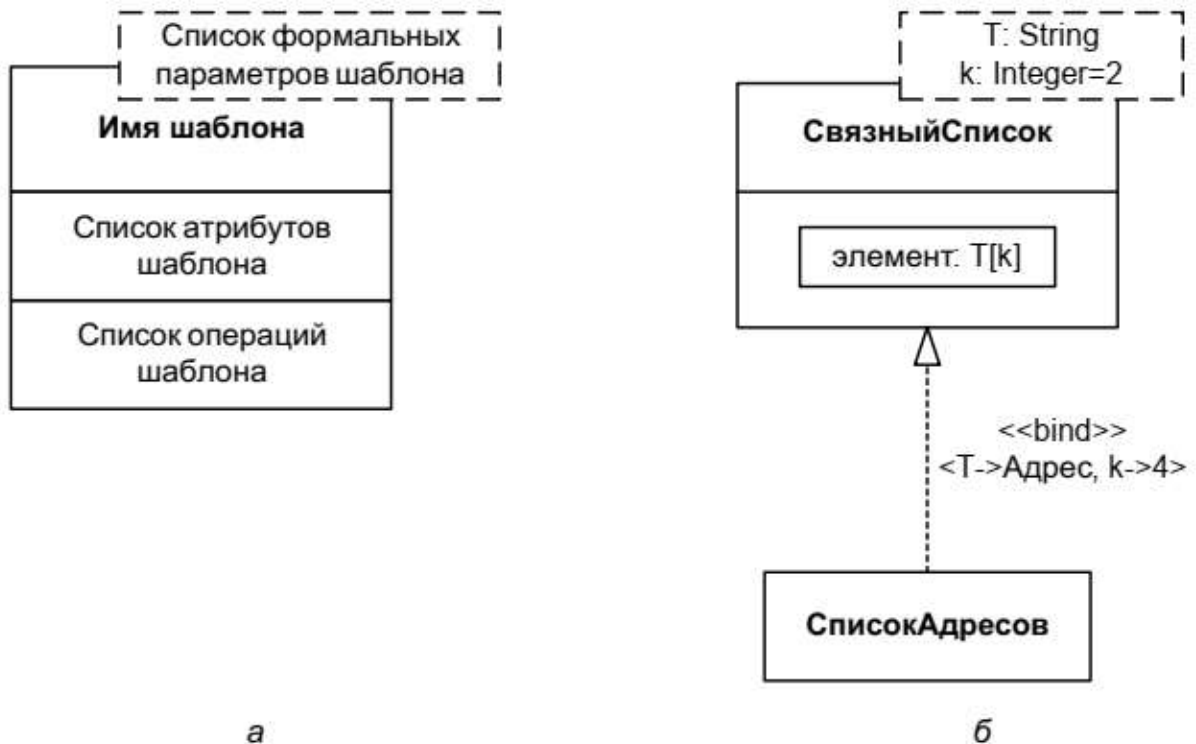


Рис. 4.24. Графическое изображение шаблона на диаграмме классов:
а — общее представление шаблона; **б** — связывание шаблона

Информация связывания в общем случае изображается в форме разделенного запятыми списка подстановок параметров шаблона:

$\langle \text{подстановка-параметра-шаблона} \rangle ::= \langle \text{имя-параметра-шаблона} \rangle ' -> ' \langle \text{действительный-параметр-шаблона} \rangle$

где синтаксис $\langle \text{имя-параметра-шаблона} \rangle$ зависит от вида формального параметра шаблона, а $\langle \text{действительный-параметр-шаблона} \rangle$ зависит от вида элемента, используемого в качестве действительного параметра.

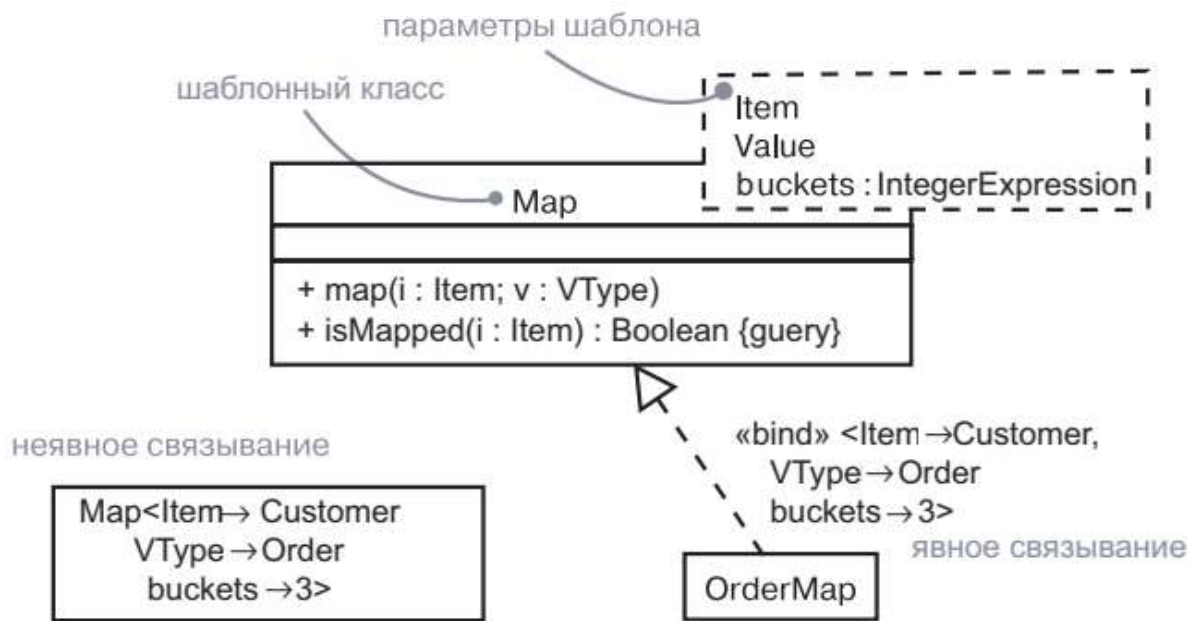
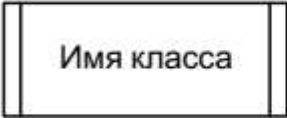
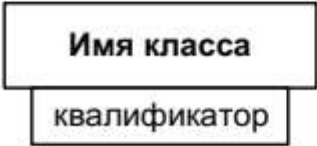
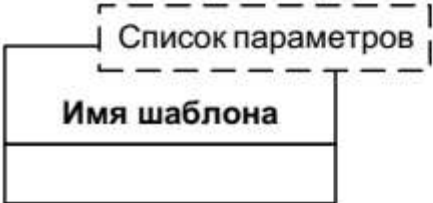
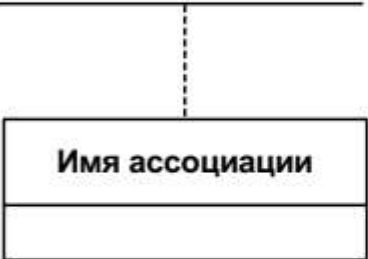


Рис. 9.7. Шаблонные классы

1.1.3. Сводная таблица

Тип графического элемента	Нотация
Класс (class) — символ классификатора	
Активный класс (active class)	
Интерфейс (interface) — специальный символ	
Интерфейс (interface) — символ классификатора	
Квалификатор (qualifier)	
Шаблон (template)	

Тип графического элемента	Нотация
Бинарная ассоциация (binary association)	<i>символ навигации</i> 
n-арная ассоциация (n-ary association)	
Ассоциация-класс (association class)	
Обобщение (generalization)	
Агрегация (aggregation)	
Композиция (composition)	
Зависимость (dependency)	
Реализация (realization)	

1.1.4. Примеры диаграмм классов

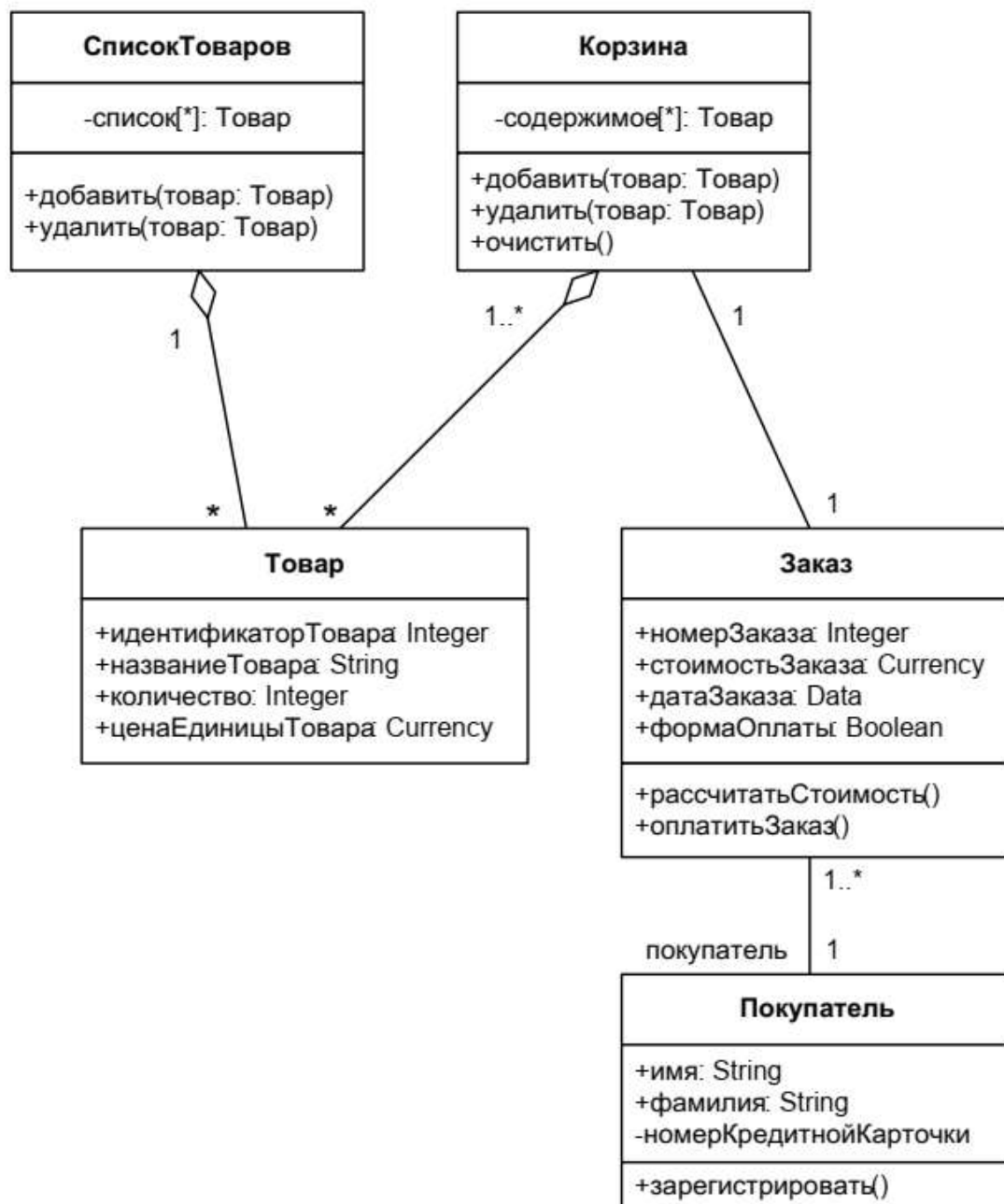


Рис. 4.25. Диаграмма классов системы продажи товаров в интернет-магазине

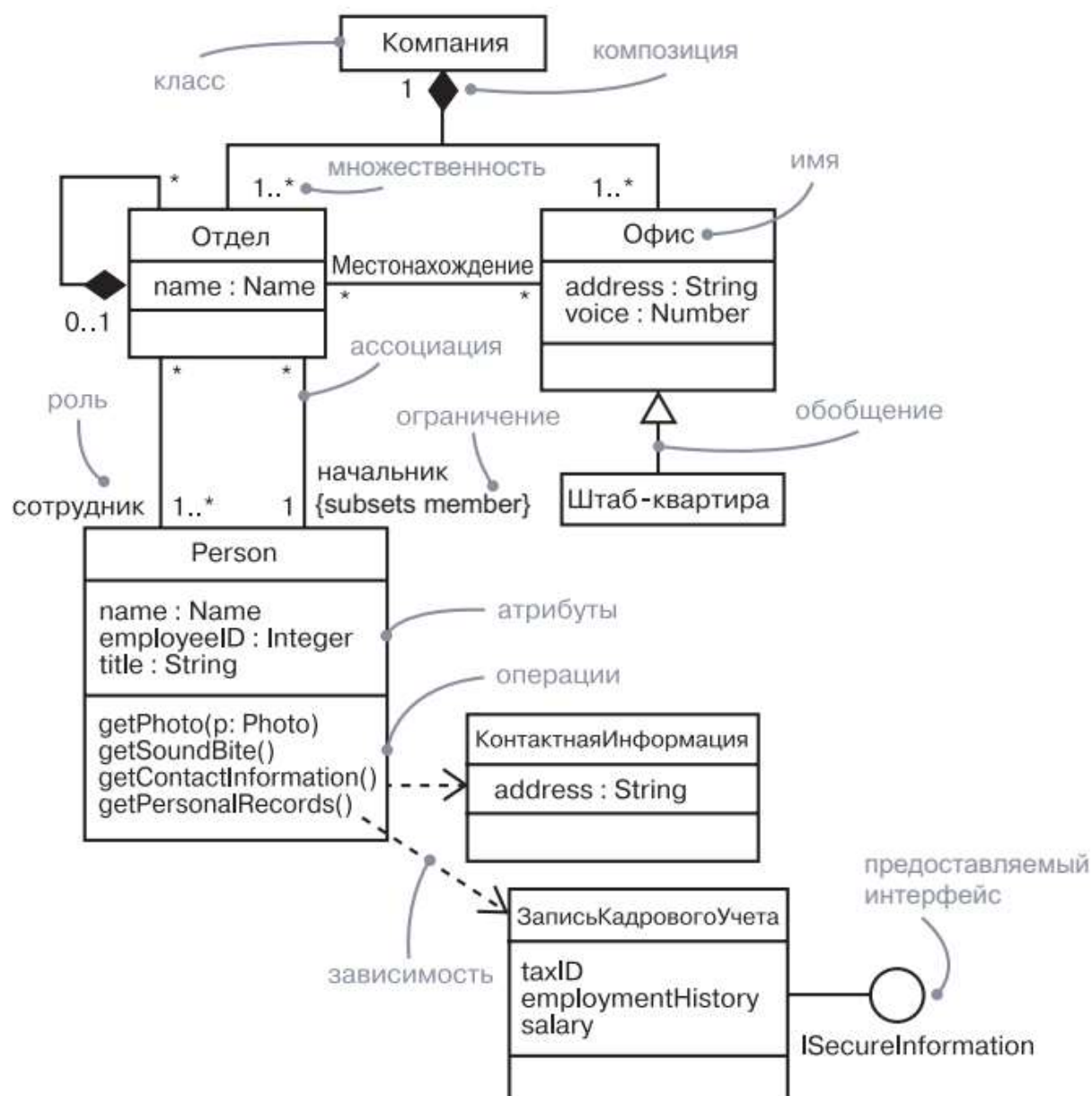


Рис. 8.1. Диаграмма классов

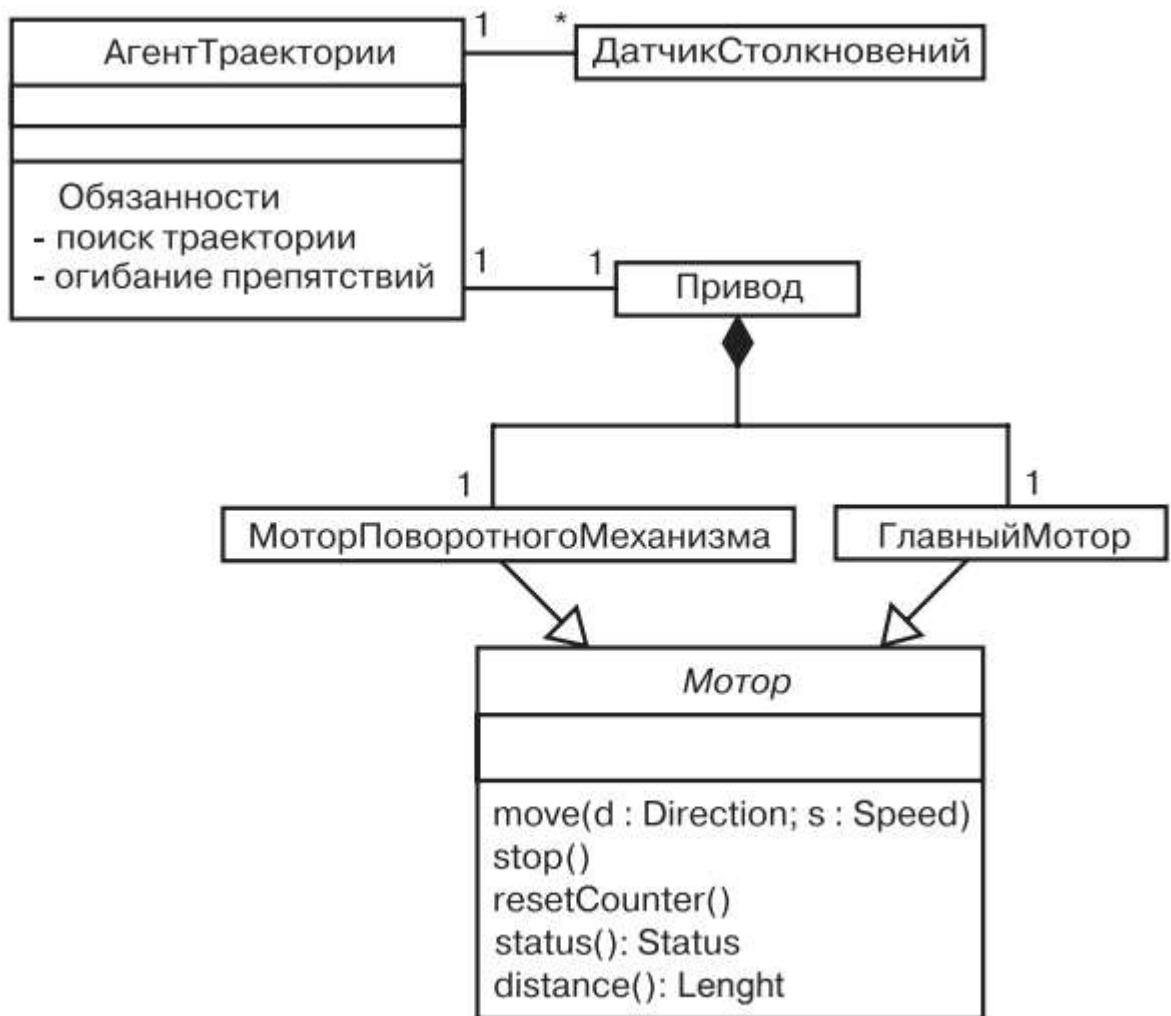


Рис. 8.2. Моделирование простой кооперации

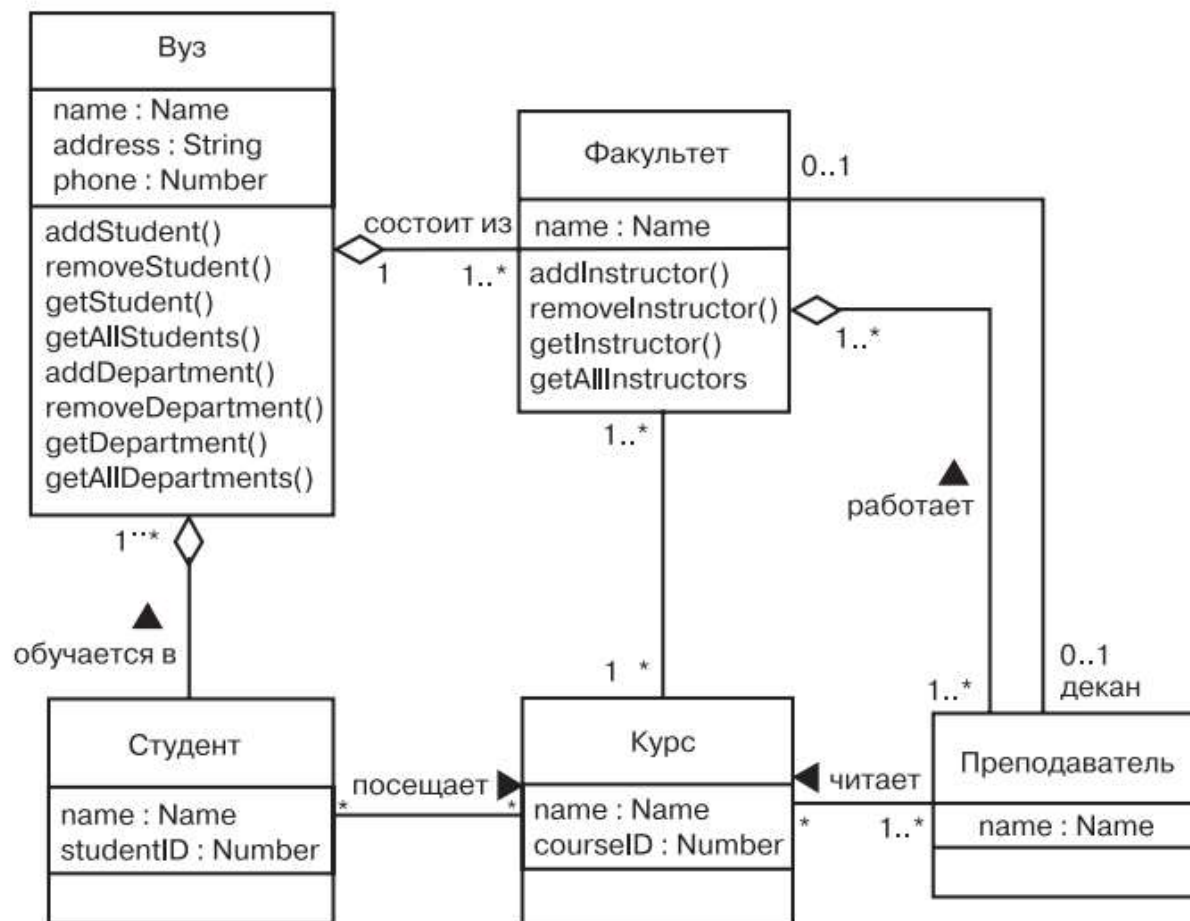
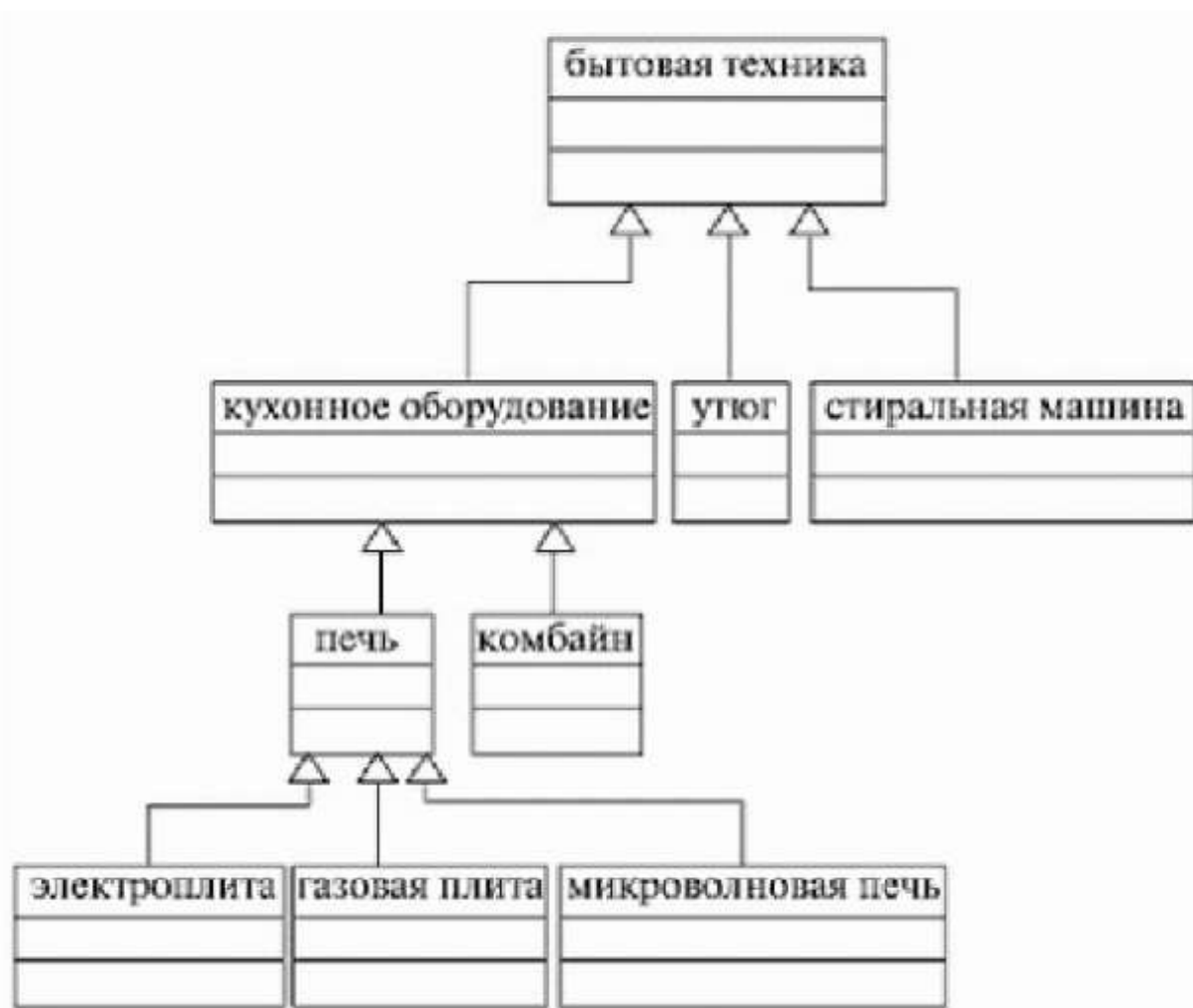
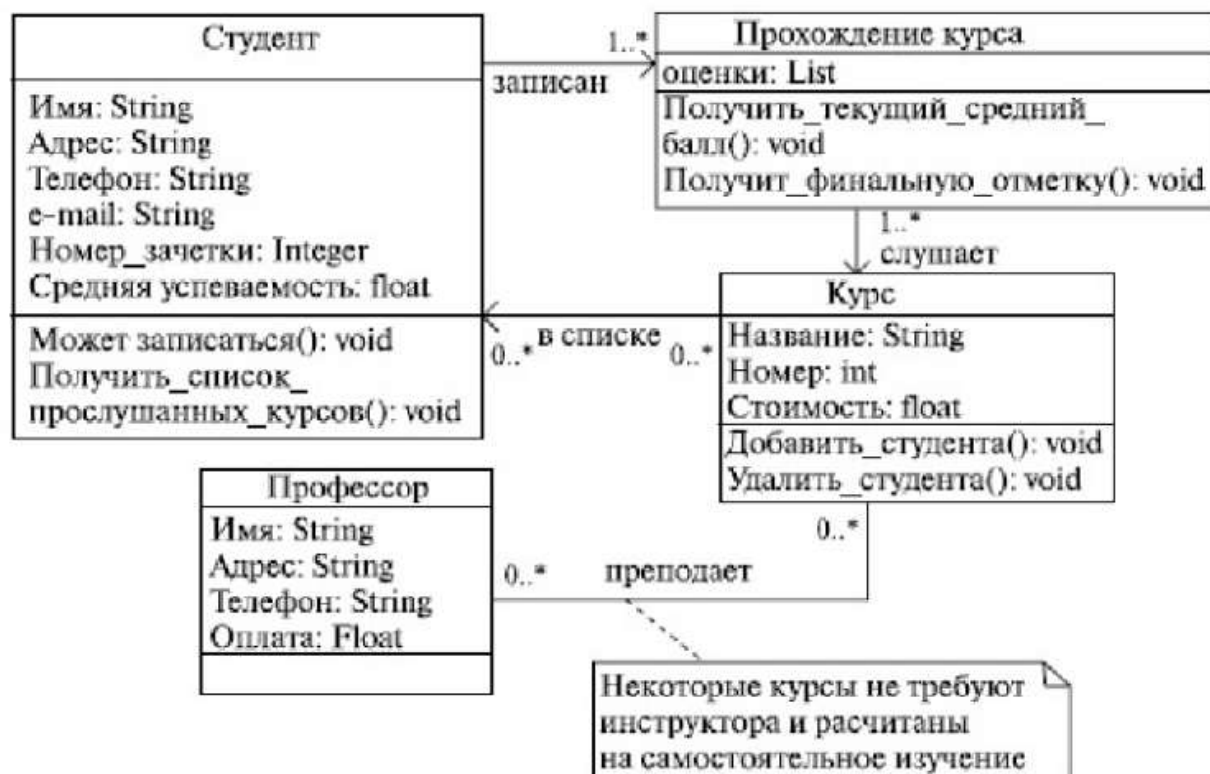


Рис. 8.3. Моделирование схемы базы данных





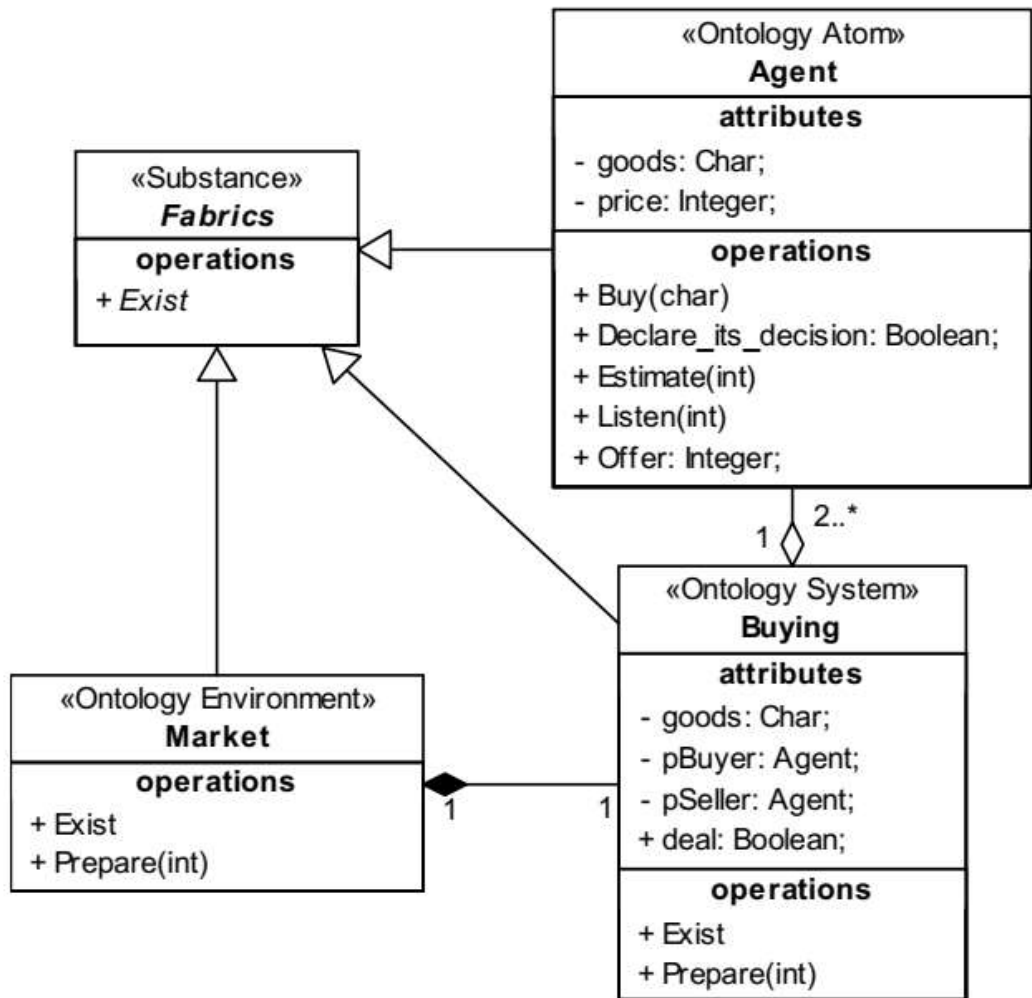


Рис. 5. Диаграмма классов имитационной модели

Первый шаг декомпозиции – контекстная диаграмма (рис. 7).

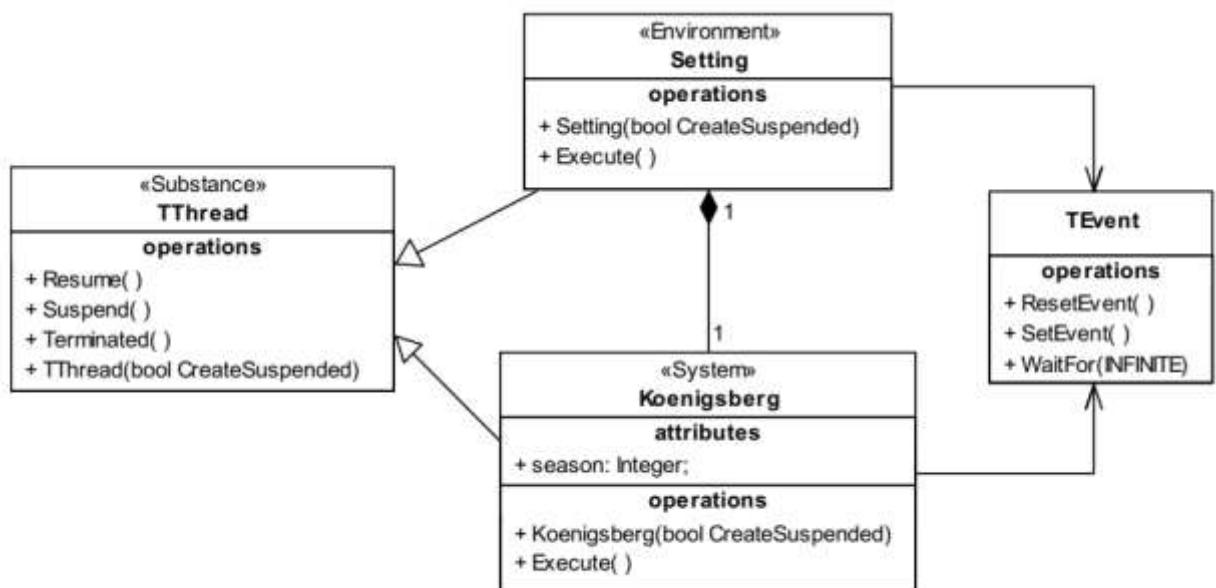


Рис. 7. Контекстная диаграмма

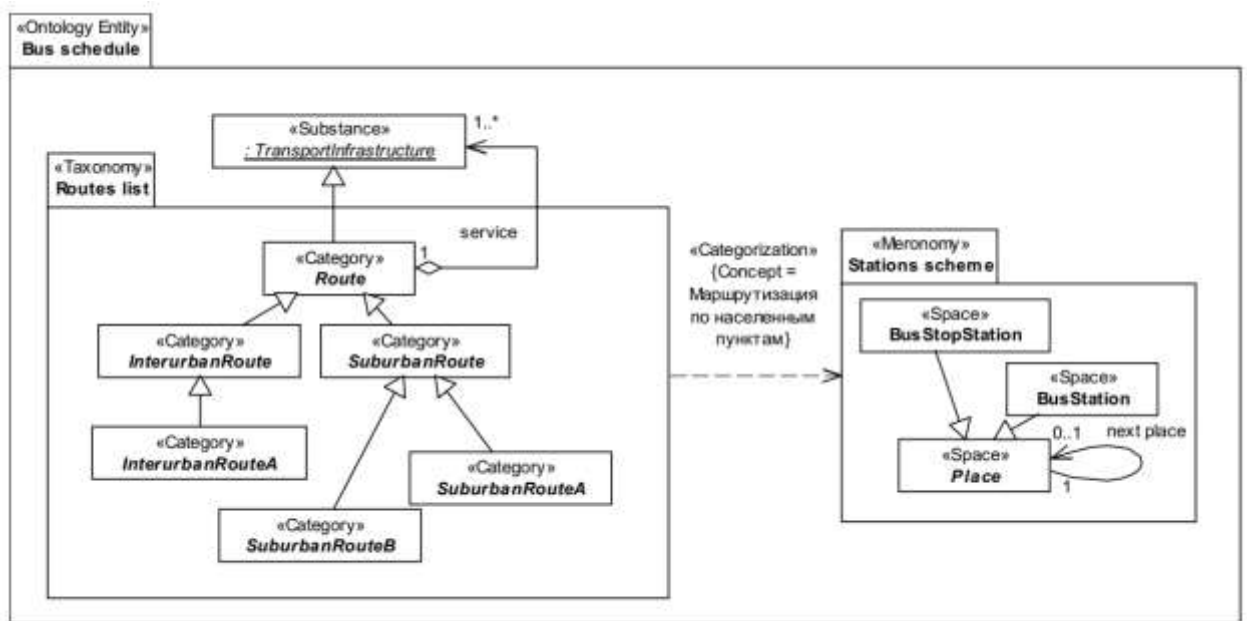


Рис. 10. Пример модели неархической системы классификации

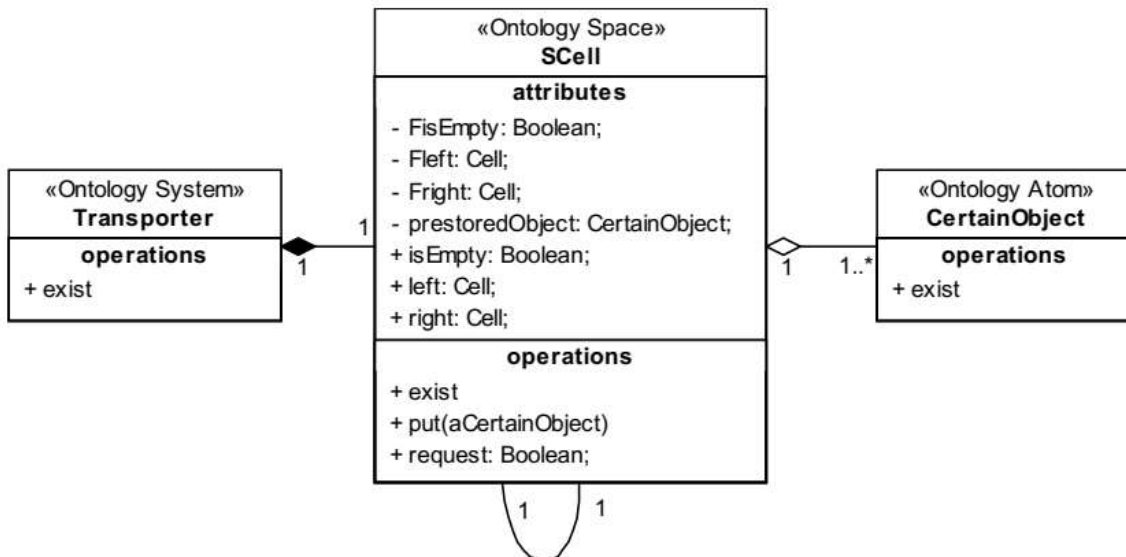


Рис. 11. Диаграмма классов для модели перемещения объектов

